

# PATENTMIND AI

Enterprise-Grade Legal Knowledge Retrieval & Strategy Platform

|                    |                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------|
| Author / Developer | Bhushan Patil (+91 93590 83546)                                                                           |
| Role               | System Architect & Full-Stack AI Engineer                                                                 |
| Tech Stack         | FastAPI, React (Vite, Framer Motion), MySQL, SQLite, Groq Cloud API                                       |
| Repository         | <a href="https://github.com/bhushanpatil3544/patentmind-ai">github.com/bhushanpatil3544/patentmind-ai</a> |
| Live URL           | <a href="https://patentmind-ai-p6qx.vercel.app">patentmind-ai-p6qx.vercel.app</a>                         |

## Executive Summary

PatentMind AI is a production-ready, highly optimized Knowledge Management and Patent Strategy platform. It leverages a dual-database routing design (MySQL primary, SQLite thread-safe fallback) and a hybrid Retrieval-Augmented Generation (RAG) architecture to search, index, and analyze complex patent specifications. The interface has been fully polished with dynamic Vercel-style aesthetics and micro-animations using Framer Motion. The backend is optimized for resource-constrained, high-throughput serverless environments.

# 1. Architecture Overview

The system follows a classic decoupled client-server architecture. The frontend is built as a Single Page Application (SPA) using React, styled with Tailwind CSS, and compiled using Vite. The backend is an asynchronous API service built with Python FastAPI. The application uses a modular layout to separate processing tasks from the main web requests.

| Layer            | Component           | Responsibility                                                                |
|------------------|---------------------|-------------------------------------------------------------------------------|
| Client Layer     | React SPA           | Renders floating sidebar, search queries, dynamic chatbot, and PDF uploads.   |
| API Gateway      | FastAPI Web Layer   | Ingests documents, runs auth rules, manages sessions, and forwards queries.   |
| RAG Orchestrator | IntelligentRAGChain | Constructs prompts, queries indices, handles local Ollama vs. Groq failovers. |
| Storage Layer    | Dual Relational DB  | MySQL (primary cloud hosting) / SQLite (read-only container fallback).        |
| Vector Store     | DualVectorStore     | ChromaDB vector embedding indexer with a keyword SQL search fallback.         |

## System Execution Workflow

The interaction process flows as follows:

- **Authentication:** Users register using Gmail OTP codes. Access tokens are generated as secure, signature-validated JWT tokens.
- **Document Ingestion:** Patented PDFs are uploaded, parsed (via PyMuPDF or PaddleOCR), chunked by section, and indexed.
- **RAG Query Flow:** When a query is made, it generates a text embedding (or uses keyword SQL fallback) to retrieve patent abstracts.
- **LLM Generation:** The prompt is compiled with relevant chunks and sent to the LLM (primary local Ollama, fallback Groq API).

## 2. Technology Stack & AI Models

To ensure high scalability and cost efficiency, the AI models and inference engines are split between a local development stack and a fast, low-latency fallback cloud setup.

| Component      | Primary Model                           | Cloud Fallback / Alternative                | Rationale                                                                                    |
|----------------|-----------------------------------------|---------------------------------------------|----------------------------------------------------------------------------------------------|
| Embeddings     | SentenceTransformers (all-MiniLM-L6-v2) | Relational SQLite Keyword Search (SQL LIKE) | MiniLM provides fast local vectors. SQL FTS bypasses memory caps in serverless environments. |
| Llama Engine   | Ollama (Llama 3.1 8B)                   | Groq Cloud (LLaMA3-8B-8192)                 | Local Llama keeps data private. Groq API is lightning-fast and has 0% hosting costs.         |
| OCR Extraction | PyMuPDF (fitz)                          | PaddleOCR (scanned pages)                   | PyMuPDF extracts digital text layouts immediately. PaddleOCR scans image layers.             |

### Why We Chose This Architecture:

**Ollama vs. Groq Cloud:** Locally, Ollama allows complete data ownership (no patent leaks). However, cloud serverless hosts (like Vercel) have tight execution timeouts (15s) and lack GPU nodes. Integrating the Groq API key allows the cloud-deployed version to respond to RAG questions in under 1 second, making the project production-viable and blazing fast for users.

**SentenceTransformers vs. SQL Keyword Search:** Loading PyTorch and local models requires ~2GB of RAM, which exceeds the limits of free serverless web hosting tiers. To prevent the cloud app from failing, we implemented a smart database search fallback that parses the query for terms and performs optimized queries on the database metadata, ensuring 100% service availability.

### 3. Production Engineering & Challenges

Transforming the prototype into a production-grade application required resolving several technical and architectural challenges:

| Challenge Faced                     | Why It Occurred                                                                                                           | Resolution Implemented                                                                                                     |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 1. Vercel Serverless Function Crash | FastAPI tried to create database directories on startup, but Vercel containers have a read-only filesystem.               | Redirected SQLite storage path to '/tmp/users_and_metadata.db' and wrapped directory creations in safe check blocks.       |
| 2. 504 Gateway Timeout Errors       | FastAPI waited 45s trying to connect to local Ollama on Vercel before timing out (Vercel kills functions after 15s).      | Optimized local Ollama connection timeout to 1s when in Vercel. Connect timeouts fail fast and redirect to Groq instantly. |
| 3. Single-Connection DB Overhead    | Creating and closing raw TCP socket connections on every database query caused latency spikes and socket pool exhaustion. | Implemented PooledConnectionWrapper and a thread-safe Queue connection pool, recycling DB handles automatically.           |
| 4. In-Memory Search Freezes         | The search fallback originally pulled all database patents into memory to loop through keywords, blocking the web loop.   | Wrote an optimized search_patents_by_keywords database routine executing SQL LIKE filters directly on the database engine. |
| 5. Persistent Cache Stale UI        | Vite js/css chunks were aggressively cached by browsers, preventing users from seeing frontend updates.                   | Configured Vercel-level cache-invalidation headers ('no-store, no-cache') on index.html within vercel.json.                |

#### System Hardening Details:

By resolving these bottlenecks, the app compiles, runs, and serves requests securely. Passwords are saved using salted bcrypt hashes, session tokens use signature-verified JWT structures, and SQL parameters are fully bound to block SQL injection attacks.

## 4. Resume Showcase Strategy & Metrics

To make this project stand out in technical interviews and on your resume, emphasize the following architectural decisions:

### Key Highlights to Mention on Your Resume:

- **Hybrid Multi-Model Engine:** Configured a fallback pipeline that automatically routes queries between local Ollama instances and Groq Cloud endpoints based on latency and network availability.
- **Thread-Safe Connection Pool:** Designed a custom connection pool manager using proxy patterns that keeps socket usage low and improves response speeds under concurrent loads.
- **Cross-Database Compatibility:** Coded a fallback pipeline that moves from MySQL (production) to local SQLite storage based on active configurations.
- **Secure JWT Authentication:** Implemented a complete registration and login pipeline using Gmail OTP codes and signature-verified JWT tokens.

### Example Interview Q&A::

*Q: How does the application scale to support multiple users on serverless environments?*

**A:** On serverless environments, database and model loading present cold-start issues. We resolved this by: (1) Moving vector embedding queries to SQLite-level LIKE queries to avoid loading 2GB models, (2) Using connection pooling to reuse active database links, and (3) Routing LLM prompts to Groq API endpoints, which complete generations in under 0.5 seconds.

*Q: Why did you choose SQLite as a database fallback instead of using only MySQL?*

**A:** SQLite provides a zero-dependency, local file-based store. If cloud MySQL links go down or are blocked by corporate firewalls, the application automatically switches to SQLite in '/tmp', allowing seamless off-grid usage without crashing.

### End of Specifications Report